

JAVA

Guia Rápido de Classes e Métodos

Math, Random, BigDecimal, java.time, Locale e ResourceBundle — todas as classes, métodos e constantes citados, organizados para consulta rápida.

Companion da Apostila Completa • Math • Random • BigDecimal • java.time • Locale • ResourceBundle

Mais de 20 classes • Consulta rápida por assunto • Série JavaStudiesHub

CONTEÚDO DA APOSTILA

- 01** Utilitários Numéricos
- 02** Datas e Horas Locais
- 03** Pontos no Tempo e Fusos
- 04** Quantidades Temporais
- 05** Formatação de Data e Hora
- 06** Localização
- 07** Internacionalização
- 08** Classes de Apoio e Legado

JavaStudiesHub

Sumário

01 **Utilitários Numéricos**
Math, Random, BigDecimal, MathContext, RoundingMode

02 **Datas e Horas Locais**
LocalDate, LocalTime, LocalDateTime

03 **Pontos no Tempo e Fusos**
Instant, ZoneId, ZoneRules, ZonedDateTime, OffsetDateTime/OffsetTime

04 **Quantidades Temporais**
Duration, Period, ChronoUnit, ChronoField, TemporalAdjusters

05 **Formatação de Data e Hora**
DateTimeFormatter, FormatStyle, especificadores %t

06 **Localização**
Locale, NumberFormat, Currency, Scanner (locale)

07 **Internacionalização**
ResourceBundle, ListResourceBundle, arquivos .properties

08 **Classes de Apoio e Legado**
System, String, TimeZone, Date, GregorianCalendar

01

Utilitários Numéricos

Math, Random, BigDecimal, MathContext, RoundingMode

java.lang.Math

Membro	Descrição
PI, E	Constantes double: π e o número de Euler
abs(x)	Valor absoluto (int/long/float/double)
max(a,b) / min(a,b)	Maior/menor entre dois valores (4 overloads cada)
pow(base, exp)	Potenciação
sqrt(x) / cbrt(x)	Raiz quadrada / raiz cúbica
round(x)	Arredondamento matemático tradicional
ceil(x) / floor(x)	Sempre arredonda para cima / para baixo
exp(x) / log(x) / log10(x)	Exponencial natural / logaritmo natural / log base 10
sin/cos/tan/asin/acos/atan	Funções trigonométricas e suas inversas
random()	double aleatório entre 0 (inclusive) e 1 (exclusivo)
incrementExact / decrementExact	+1 / -1 com ArithmeticException em overflow
addExact / subtractExact / multiplyExact	Operação com ArithmeticException em overflow
absExact(x)	Valor absoluto com exceção em vez de overflow silencioso

java.util.Random

Membro	Descrição
<code>new Random()</code>	Semente baseada em <code>nanotime</code> + <code>seedUniquifier</code>
<code>new Random(long seed)</code>	Semente fixa — sequência reproduzível
<code>setSeed(long)</code>	Reinicia o estado interno com uma nova semente
<code>nextInt()</code> / <code>nextInt(bound)</code>	<code>int</code> aleatório; com <code>bound</code> , 0 (inclusive) até <code>bound</code> (exclusivo)
<code>nextInt(origin, bound)</code>	JDK 17+: <code>origin</code> (inclusive) até <code>bound</code> (exclusivo)
<code>nextDouble</code> / <code>nextBoolean</code> / <code>nextLong</code> / <code>nextFloat</code>	Valores aleatórios dos respectivos tipos
<code>ints()</code> / <code>ints(size)</code>	Stream infinito; ou finito de tamanho <code>size</code> , sem bounds
<code>ints(origin, bound)</code>	Stream infinito dentro do intervalo (precisa de <code>limit</code>)
<code>ints(size, origin, bound)</code>	Stream finito de tamanho <code>size</code> , dentro do intervalo
<code>doubles(...)</code> / <code>longs(...)</code>	Mesmos padrões de overload de <code>ints</code> , para <code>double/long</code>
<code>RandomGenerator</code>	Interface implementada por <code>Random</code> desde o JDK 17

01

Utilitários Numéricos (continuação)

BigDecimal, MathContext, RoundingMode

java.math.BigDecimal

Membro	Descrição
<code>new BigDecimal(String)</code>	Preserva o valor exatamente como escrito — construtor recomendado
<code>new BigDecimal(double)</code>	Herda imprecisão do <code>double</code> — desencorajado
<code>BigDecimal.valueOf(double)</code>	Atalho via <code>Double.toString()</code> ; mais seguro que o construtor de <code>double</code>
<code>ONE / ZERO / TEN</code>	Constantes estáticas prontas
<code>unscaledValue()</code>	<code>BigInteger</code> com todos os dígitos, sem posição do ponto decimal
<code>scale()</code>	Número de dígitos à direita do ponto decimal
<code>precision()</code>	Número total de dígitos significativos
<code>setScale(int)</code>	Ajusta a escala; lança exceção se precisar arredondar sem <code>RoundingMode</code>
<code>setScale(int, RoundingMode)</code>	Ajusta a escala com modo de arredondamento explícito
<code>add / subtract / multiply / divide</code>	Operações aritméticas — todas retornam nova instância
<code>divide(BigDecimal, MathContext)</code>	Divisão controlando precisão e arredondamento do resultado
<code>abs()</code>	Valor absoluto
<code>compareTo / equals</code>	Comparação numérica / comparação de valor e escala

java.math.MathContext

Membro	Descrição
<code>UNLIMITED</code>	Sem limite de precisão (não resolve dízimas periódicas)
<code>DECIMAL32 / 64 / 128</code>	7 / 16 / 34 dígitos significativos, <code>RoundingMode.HALF_EVEN</code>
<code>new MathContext(precision, mode)</code>	Contexto customizado com precisão e arredondamento à escolha

java.math.RoundingMode (enum)

Constante	Comportamento
HALF_UP	0,5+ no dígito descartado arredonda para cima
HALF_DOWN	0,5+ no dígito descartado arredonda para baixo
HALF_EVEN	Arredonda para o vizinho par
UP / DOWN	Sempre se afasta de zero / sempre se aproxima de zero
CEILING / FLOOR	Sempre em direção a $+\infty$ / sempre em direção a $-\infty$
UNNECESSARY	Lança exceção se arredondamento for necessário

02

Datas e Horas Locais*LocalDate, LocalTime, LocalDateTime***java.time.LocalDate**

Membro	Descrição
now() / of(y,m,d) / parse(String)	Data atual / instância explícita / a partir de String ISO
EPOCH	Constante: 1970-01-01
getYear / getMonth / getMonthValue	Ano; mês como enum Month; mês como int
getDayOfMonth	Dia do mês — valor armazenado diretamente
getDayOfWeek / getDayOfYear	Dia da semana / dia do ano — calculados, não armazenados
get(TemporalField)	Valor de um campo via ChronoField (ex: MONTH_OF_YEAR)
withYear / withMonth / withDayOfMonth / withDayOfYear	Nova instância com o campo alterado
with(TemporalField, valor) / with(TemporalAdjuster)	Versões genéricas de with
plusYears / plusMonths / plusDays / plusWeeks	Nova instância somando a unidade indicada
plus(qtd, TemporalUnit) / minus(...)	Versão genérica de plus/minus via ChronoUnit
isAfter / isBefore / isEqual	Comparação booleana entre datas
compareTo / equals	Comparação ordinal / comparação de igualdade
isLeapYear()	Verifica se o ano é bissexto
range(TemporalField)	ValueRange com os limites válidos de um campo
datesUntil(LocalDate)	Stream até o limite (exclusivo), passo de 1 dia
datesUntil(LocalDate, Period)	Mesmo stream, com passo customizado
atStartOfDay() / atStartOfDay(Zoneld)	Combina com LocalTime.MIDNIGHT / gera ZonedDateTime
atTime(LocalTime)	Combina com uma hora, gerando LocalDateTime
format(DateTimeFormatter)	Formata a data conforme o formatter

02

Datas e Horas Locais (continuação)*LocalTime, LocalDateTime***java.time.LocalTime**

Membro	Descrição
now() / of(h,m) / of(h,m,s) / parse(String)	Hora atual / instâncias explícitas / a partir de String
Armazenamento	Horas em formato 24h (0–23); byte para h/m/s, int para nanos
getHour / getMinute / getSecond / getNano	Getters dos componentes de hora
get(ChronoField.AMPM_OF_DAY)	0 = antes do meio-dia, 1 = depois do meio-dia
withHour / withMinute / withSecond	Nova instância com o campo alterado
plusHours / plusMinutes / plusSeconds	Nova instância somando a unidade indicada
range(TemporalField)	Ex.: HOUR_OF_DAY → 0–23; MINUTE_OF_HOUR → 0–59
Campos/unidades não suportados	ChronoField.YEAR e ChronoUnit.DAYS lançam UnsupportedOperationException

java.time.LocalDateTime

Membro	Descrição
now() / of(LocalDate, LocalTime) / of(y,m,d,h,min)	Combinações de criação
ofInstant(Instant, ZoneId)	Converte um Instant para data-hora local em um fuso
toLocalDate() / toLocalTime()	Extraí a parte de data / a parte de hora
Métodos herdados	Todos os métodos get/with/plus/minus de LocalDate e LocalTime
format(DateTimeFormatter)	Formata data e hora combinadas

03

Pontos no Tempo e Fusos Horários

Instant, ZoneId, ZoneRules, ZonedDateTime, OffsetDateTime/OffsetTime

java.time.Instant

Membro	Descrição
now() / parse(String) / EPOCH	Momento atual / a partir de String ISO_INSTANT / 1970-01-01T00:00Z
getEpochSecond() / getNano()	Segundos e nanossegundos desde a epoch
Sempre referenciado a UTC	Sufixo Z na saída; não depende do fuso do sistema

java.time.ZoneId

Membro	Descrição
systemDefault()	Fuso horário atual da JVM
of(String id)	Instância a partir de um identificador Continente/Cidade
of(String id, Map)	Resolve códigos antigos via um mapa de sinônimos
SHORT_IDS	Campo estático: mapa de códigos antigos (ex: "BET") para IDs atuais
getAvailableZoneIds()	Set com todos os identificadores suportados (600+)
getRules()	Devolve um ZoneRules com as regras daquele fuso
Formato do identificador	Continente/Cidade; underscore no lugar de espaço (Broken_Hill)

java.time.zone.ZoneRules

Membro	Descrição
getOffset(Instant)	Offset UTC vigente naquele instante
isDaylightSavings(Instant)	boolean: horário de verão em vigor naquele instante?
getDaylightSavings(Instant)	Duration do ajuste (ex: PT0S ou PT1H)

03

Pontos no Tempo e Fusos (continuação)

ZonedDateTime, OffsetDateTime, OffsetTime

java.time.ZonedDateTime

Membro	Descrição
now(ZoneId) / ofInstant(Instant, ZoneId)	Instância atual em um fuso / a partir de um Instant
atZone(ZoneId)	A partir de um LocalDateTime, combina com um fuso
withZoneSameInstant(ZoneId)	Mesmo instante, hora local recalculada para o novo fuso
withZoneSameLocal(ZoneId)	Mesmos números de data-hora, instante diferente
with(TemporalAdjuster)	Aplica um ajuste predefinido (ver TemporalAdjusters)
toLocalDate / toLocalDateTime / toInstant	Extraí a parte correspondente
Métodos herdados	Todos os get/with/plus/minus de LocalDateTime

OffsetDateTime / OffsetTime

Combinam LocalDateTime/LocalTime com um offset UTC fixo, sem nome de região e sem aplicar regras de horário de verão — diferente de ZonedDateTime, que sempre resolve dinamicamente contra as regras da região.

04

Quantidades Temporais

Duration, Period, ChronoUnit, ChronoField, TemporalAdjusters

java.time.Duration

Membro	Descrição
ofHours / ofMinutes / ofSeconds / ofDays	Cria uma Duration a partir de uma única unidade
between(Temporal, Temporal)	Duration entre dois pontos, baseada em tempo de relógio
plusHours / plusMinutes / plusSeconds	Nova Duration somando a unidade indicada
toHours / toMinutes / toSeconds	Duração total convertida para a unidade indicada
toHoursPart / toMinutesPart / toSecondsPart	Parte 'restante' de cada unidade (ex: 2h 15min → 15 em toMinutesPart)
Saída textual	Formato PTnHnMnS (Period of Time)

java.time.Period

Membro	Descrição
ofDays / ofMonths / ofYears	Cria um Period a partir de uma única unidade
between(LocalDate, LocalDate)	Period entre duas datas, em anos/meses/dias
Saída textual	Formato PnYnMnD (ex: P3M29D = 3 meses e 29 dias)

java.time.temporal.ChronoUnit (enum)

Membro	Descrição
NANOS / MICROS / MILLIS / SECONDS / MINUTES / HOURS	Unidades baseadas em relógio
DAYS / WEEKS / MONTHS / YEARS / DECADES / CENTURIES / MILLENNIA	Unidades baseadas em calendário
between(Temporal, Temporal)	long: quantidade dessa unidade entre dois pontos
isSupportedBy(Temporal)	Verifica se a unidade é válida para essa classe antes de usar
Uso em plus/minus/until	Parâmetro TemporalUnit desses métodos

04

Quantidades Temporais (continuação)

ChronoField, TemporalAdjusters

java.time.temporal.ChronoField (enum)

Membro	Descrição
YEAR / MONTH_OF_YEAR / DAY_OF_MONTH / DAY_OF_YEAR	Campos de data
DAY_OF_WEEK	Campo de dia da semana
HOUR_OF_DAY / MINUTE_OF_HOUR / SECOND_OF_MINUTE / SECOND_OF_DAY	Campos de hora
AMPM_OF_DAY	0 (antes do meio-dia) ou 1 (depois do meio-dia)
Uso em get/with	Parâmetro TemporalField desses métodos

java.time.temporal.TemporalAdjusters

Membro	Descrição
firstDayOfMonth() / firstDayOfNextMonth()	Primeiro dia do mês atual / do próximo mês
lastDayOfMonth() / lastDayOfYear()	Último dia do mês atual / do ano atual
next(DayOfWeek) / previous(DayOfWeek)	Próxima / anterior ocorrência de um dia da semana
Uso	Passado como argumento para with(TemporalAdjuster)

05

Formatação de Data e Hora

DateTimeFormatter, FormatStyle, especificadores %t

java.time.format.DateTimeFormatter

Membro	Descrição
ISO_DATE / ISO_INSTANT / ISO_WEEK_DATE	Formatadores padrão prontos, como campos estáticos
ofPattern(String)	Formatador a partir de um padrão customizado (EEEE, MMMM, d, yyyy...)
ofLocalizedDate(FormatStyle)	Formatador de data, estilo predefinido, sensível a Locale
ofLocalizedDateTime(FormatStyle)	Formatador de data e hora combinadas
withLocale(Locale)	Nova instância do formatter, adaptada a um Locale
format(TemporalAccessor)	Aplica o formatter a uma data/hora

java.time.format.FormatStyle (enum)

Constante	Descrição
FULL	Forma mais completa e textual (ex: quarta-feira, 1 de julho de 2026)
LONG	Forma longa, geralmente sem o dia da semana
MEDIUM	Forma intermediária, comum para uso geral
SHORT	Forma mais compacta, geralmente numérica

Padrões de ofPattern (letras mais comuns)

Letra	Significado
E (EEEE)	Dia da semana — abreviado / completo com 4 repetições
M (MMMM)	Mês — numérico ou 2 dígitos / nome completo com 4 repetições
d	Dia do mês, numérico
y ou u (yyyy)	Ano — 2 dígitos ou completo com 4 repetições

05

Formatação (continuação)*Especificadores %t de printf / String.format***Especificadores %t em printf / String.format**

Especificador	Saída
%tD	Data no formato MM/dd/aa
%tF	Data no formato ISO (aaaa-MM-dd)
%tr	Hora no formato 12h com AM/PM
%tT	Hora no formato 24h (HH:mm:ss)
%tA / %ta	Dia da semana completo / abreviado
%tB / %tb	Mês completo / abreviado
%te	Dia do mês sem zero à esquerda
%tY / %ty	Ano com 4 dígitos / 2 dígitos
1\$ (índice posicional)	Reaproveita o mesmo argumento em múltiplas conversões %t
Locale como 1º argumento	printf(Locale, formato, args) / String.format(Locale, ...)

06

Localização

Locale, NumberFormat, Currency, Scanner (locale)

java.util.Locale

Membro	Descrição
getDefault() / setDefault(Locale)	Consulta / sobrescreve o Locale padrão da JVM
new Locale(idioma) / new Locale(idioma, país)	Construtores — idioma é obrigatório
forLanguageTag(String)	Instância a partir de uma language tag (ex: "en-AU")
Locale.Builder	setLanguage/setRegion/build — construção fluente
Constantes de idioma	ENGLISH, FRENCH, GERMAN, etc.
Constantes de país	US, UK, CANADA, CANADA_FRENCH, FRANCE, GERMANY, JAPAN, etc.
getDisplayName() / getDisplayName(Locale)	Nome do locale no idioma padrão / em outro idioma
getLanguage() / getCountry()	Código de idioma (ISO 639) / código de país (ISO 3166)
Campos do Locale	language, country/region, variant, script, extensions

java.text.NumberFormat

Membro	Descrição
getNumberInstance(Locale)	Formatador de números decimais comuns
getCurrencyInstance(Locale)	Formatador de valores monetários, com símbolo e arredondamento
format(double)	Aplica a formatação a um valor
setMaximumFractionDigits(int)	Ajusta casas decimais (classe mutável)

06

Localização (continuação)

Currency, Scanner sensível a Locale

java.util.Currency

Membro	Descrição
getInstance(Locale)	Moeda associada a um Locale
getCurrencyCode()	Código ISO de 3 letras (ex: "BRL", "JPY")
getDisplayName() / getDisplayName(Locale)	Nome textual da moeda, no idioma padrão / em outro idioma

java.util.Scanner — Métodos Sensíveis a Locale

Membro	Descrição
useLocale(Locale)	Define o Locale usado para interpretar entrada numérica
nextBigDecimal()	Lê a próxima entrada como BigDecimal, respeitando separadores do Locale

07

Internacionalização

ResourceBundle, ListResourceBundle, arquivos .properties

java.util.ResourceBundle

Membro	Descrição
getBundle(String baseName)	Bundle usando o Locale padrão da JVM
getBundle(String baseName, Locale)	Bundle para um Locale específico
getString(String key)	Valor associado à chave — sensível a maiúsculas/minúsculas
keySet()	Todas as chaves disponíveis no bundle resolvido
getBaseBundleName()	Nome base configurado para o bundle
ListResourceBundle	Subclasse abstrata para dados definidos em código (não-String)
ResourceBundle.Control	Extensão para suportar formatos alternativos (ex: XML)

Arquivos .properties

Convenção	Descrição
nomeBase.properties	Arquivo raiz — usado como fallback final
nomeBase_idioma.properties	Variação por idioma (ex: TextMessages_es.properties)
nomeBase_idioma_PAÍS.properties	Variação por idioma + país (ex: _pt_BR)
chave=valor / chave: valor / chave valor	Formatos aceitos para pares chave-valor
# ou !	Início de comentário
Cadeia de fallback	Do mais específico (idioma+script+país+variante) até o arquivo base
Fallback por chave	Uma chave ausente em um arquivo específico é buscada no arquivo base

String — Parametrização de Mensagens

Membro	Descrição
formatted(Object...)	Aplica argumentos a uma String usada como molde de formatação
String.format(Locale, formato, args)	Formata com Locale explícito

08

Classes de Apoio e Legado

System, String, TimeZone, Date, GregorianCalendar

java.lang.System

Membro	Descrição
currentTimeMillis()	Milissegundos desde a epoch (1970-01-01 UTC), baseado no SO
nanoTime()	Nanossegundos desde uma origem arbitrária — só para medir tempo decorrido na JVM
setProperty("user.timezone", id)	Sobrescreve o fuso horário padrão da JVM (deve ser a 1ª instrução)

Classes Anteriores ao JDK 8 (uso desencorajado)

Classe	Substituída por
java.util.Date	Instant ou LocalDateTime
java.util.TimeZone	java.time.ZoneId
java.util.GregorianCalendar	LocalDate / LocalDateTime / ZonedDateTime
java.text.DateFormat	java.time.format.DateTimeFormatter
java.text.SimpleDateFormat	DateTimeFormatter.ofPattern

◆ Atenção

Essas classes legadas continuam funcionando e aparecem com frequência em código antigo, mas não são thread-safe (ao contrário de todo o pacote java.time) e têm APIs mais confusas. Prefira sempre as classes de java.time em código novo.

javax.swing.JOptionPane (uso pontual em exemplos de i18n)

Membro	Descrição
showOptionDialog(...)	Caixa de diálogo genérica — mensagem, título e botões podem vir de um ResourceBundle
showMessageDialog / showConfirmDialog	Variações mais simples, que internamente chamam showOptionDialog

Fim da Referência Rápida — este guia acompanha a Apostila Completa de Math, Random, BigDecimal, java.time e Localization. Para explicações, exemplos de código e contexto de uso de cada item listado aqui, consulte os capítulos correspondentes na apostila principal.